

Node.js

10-Page Concepts & Interview Guide

From runtime fundamentals to Express, MongoDB, authentication, streams, testing, deployment, and production practices.

Goal: understand what Node.js concepts are, why they are used, and how they fit together in a MERN backend.

1. Node.js Fundamentals

What is Node.js?

- Node.js is a JavaScript runtime built on Chrome's V8 engine. It lets JavaScript run outside the browser, especially on servers.
- Node is not a framework. Express, Fastify and NestJS are frameworks or libraries commonly used with it.
- Typical uses: REST APIs, real-time apps, microservices, CLI tools, file processing and backend services.

Why Node is fast for I/O

- Node uses an event-driven, non-blocking I/O model. While the program waits for a database, file system or network operation, the main JavaScript thread can continue handling other work.
- V8 executes JavaScript; Node also provides APIs for HTTP, files, networking, crypto, processes and more.

Core terms

- Runtime: environment that executes JavaScript.
- Process: the running Node application; `process.env`, `process.argv` and `process.exit()` are common APIs.
- Module: reusable unit of code. Node supports CommonJS and ES Modules.
- npm: package manager and ecosystem for Node packages.

Example

```
console.log('Hello Node');  
console.log(process.version);
```

2. Event Loop & Asynchronous JavaScript

Synchronous vs asynchronous

- Synchronous code blocks until it finishes.
- Asynchronous code starts an operation and handles its result later using callbacks, Promises or async/await.

Event loop

- The event loop coordinates JavaScript execution with asynchronous operations. Completed I/O work is queued so its callbacks can run when the call stack is available.
- Node is single-threaded for JavaScript execution, but underlying system facilities and Node's worker pool can handle many I/O and some CPU-related operations.

Promises and async/await

- A Promise represents a future result: pending, fulfilled or rejected.
- async functions return Promises. await pauses that async function until the Promise settles; it does not freeze the entire Node process.

Example

```
const data = await fs.promises.readFile('data.txt', 'utf8');
console.log(data);
```

Interview point

- Avoid heavy CPU work on the main thread. CPU-bound tasks can delay other requests; consider worker_threads, child processes or moving the work to another service.

3. Modules, npm & Project Structure

CommonJS

- Uses `require()` and `module.exports`.

```
const express = require('express');  
module.exports = router;
```

ES Modules

- Uses `import/export`. Package configuration can use `type: module`.

```
import express from 'express';  
export default router;
```

npm concepts

- `package.json` describes the project, scripts, dependencies and metadata.
- dependencies are needed by the application; `devDependencies` are normally development-only.
- `package-lock.json` locks dependency versions for reproducible installs.
- `npm install` installs dependencies; `npm run <script>` executes a package script.

Environment variables

- Use `process.env` for configuration such as `PORT`, database URLs and API keys. Never commit secrets to Git.
- Use a `.env` loader such as `dotenv` when appropriate, while production platforms often provide environment variables directly.

Useful structure

- routes → endpoints; controllers → request/response logic; models → database schemas; services → business logic; middleware → cross-cutting request processing.

4. HTTP, REST APIs & Express

HTTP basics

- Request contains method, URL, headers and optionally a body. Response contains status code, headers and body.
- Common methods: GET read, POST create/action, PUT replace, PATCH partial update, DELETE remove.
- Common status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthenticated, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Server Error.

Express

- Express is a minimal web framework for Node.js. It provides routing, middleware and request/response helpers.

Basic API

```
const express = require('express');
const app = express();
app.use(express.json());
app.get('/api/users', (req, res) => res.json([]));
app.listen(3000);
```

Request data

- req.params → URL parameters such as /users/:id.
- req.query → query string such as ?page=2.
- req.body → JSON or form body after the appropriate body parser middleware.
- res.status(...).json(...) sends a JSON response.

5. Middleware, Routing & Error Handling

Middleware

- Middleware receives req, res and next. It can inspect or modify the request, end the response, or pass control onward with next().
- Common uses: authentication, logging, validation, CORS, parsing, rate limiting and error handling.

Router

- express.Router() groups related routes and keeps the application modular.

```
router.get('/:id', getUser);  
router.post('/', createUser);  
app.use('/api/users', router);
```

Error handling

- Centralize errors with an Express error middleware having four parameters: err, req, res, next.
- Always return a useful status code and avoid leaking stack traces or secrets in production.

Async errors

- Handle rejected Promises with try/catch in async handlers or an async-wrapper pattern. Make sure errors reach the central error handler.

CORS

- CORS controls which browser origins can call your API. It is a browser security policy, not a replacement for authentication.

6. MongoDB, Mongoose & CRUD

MongoDB

- MongoDB is a document database. Data is stored as BSON documents in collections.
- CRUD means Create, Read, Update and Delete.

Mongoose

- Mongoose is an ODM for MongoDB. It provides schemas, models, validation, middleware and convenient query APIs.

Connection

```
await mongoose.connect(process.env.MONGO_URI);
```

Typical operations

```
await User.create({ name: 'A' });  
await User.find();  
await User.findById(id);  
await User.findByIdAndUpdate(id, data, { new: true });  
await User.findByIdAndDelete(id);
```

Important concepts

- Schema defines document shape and validation rules.
- Model is compiled from a schema and used to interact with a collection.
- Indexes improve query performance but consume storage and write overhead.
- `populate()` can resolve referenced documents; embedding is often better when related data is small and tightly coupled.

7. Authentication, Authorization & Security

Authentication vs authorization

- Authentication asks: who are you? Authorization asks: what are you allowed to do?

Password security

- Never store plain-text passwords. Hash passwords with a strong password-hashing algorithm such as bcrypt or Argon2 and compare hashes during login.

JWT

- A JSON Web Token can carry signed claims used for stateless authentication. The server verifies the signature before trusting claims.
- JWT is not encryption by default. Do not put secrets or sensitive information in its payload.

Security checklist

- Validate and sanitize input.
- Use HTTPS in production.
- Keep secrets in environment variables or secret managers.
- Use rate limiting for sensitive endpoints.
- Configure CORS deliberately.
- Use secure cookie settings when using cookies: HttpOnly, Secure and appropriate SameSite.
- Prevent injection by using parameterized or ODM queries and avoiding unsafe dynamic query construction.

8. Files, Streams, Buffers & Events

File system

- The `fs` and `fs/promises` modules provide file operations. Prefer asynchronous APIs for server workloads.

Buffers

- A `Buffer` represents raw binary data. It is common when handling files, network packets and encoded data.

Streams

- Streams process data in chunks instead of loading everything into memory. Types include `Readable`, `Writable`, `Duplex` and `Transform`.
- Streams are useful for large files, uploads, downloads and data transformations.

Example

```
const fs = require('fs');
const stream = fs.createReadStream('large.pdf');
stream.on('data', chunk => console.log(chunk.length));
```

EventEmitter

- Node's `events` module provides `EventEmitter`. Objects can emit named events and listeners respond to them.

```
emitter.on('done', () => console.log('finished'));
emitter.emit('done');
```

9. Advanced Node Concepts

process and child_process

- process exposes runtime information and controls. child_process can start external processes.
- Use child processes carefully and never pass untrusted input directly into shell commands.

worker_threads

- Worker threads allow JavaScript to run CPU-heavy work away from the main event-loop thread. They are useful for CPU-bound tasks, not normally necessary for ordinary I/O.

Cluster and scaling

- Node applications can use multiple processes to use multiple CPU cores. In modern deployments, process managers, containers and orchestrators are also common ways to scale.

Caching & queues

- Caching can reduce repeated database or API work. Redis is commonly used for caching, sessions and queues.
- Queues such as BullMQ allow slow or background work—emails, PDF processing, reports—to happen outside the request-response path.

WebSockets

- WebSockets provide persistent, two-way communication and are useful for chat, live notifications and real-time dashboards.

Testing & debugging

- Use Node's built-in test runner or tools such as Jest or Vitest. Use logging and the Node inspector to debug. Test routes, services, validation, authentication and failure cases.

10. Production, Deployment & Interview Checklist

Production essentials

- Use environment-specific configuration.
- Validate all external input.
- Add structured logging and meaningful error handling.
- Set timeouts and handle database or network failures.
- Use indexes and pagination for large datasets.
- Gracefully handle SIGTERM and SIGINT so deployments can shut down cleanly.
- Keep dependencies updated and audit for vulnerabilities.
- Use a reverse proxy or load balancer or managed platform as appropriate.

Typical MERN flow

- React → HTTP request → Express route → middleware → controller → service/model → MongoDB → response → React.
- For authentication: login → verify password → issue session/JWT → client sends credentials on protected requests → auth middleware verifies → controller runs.

10 interview questions to know

1. What is Node.js and why is it non-blocking?
2. Explain the event loop.
3. Difference between callbacks, Promises and async/await?
4. What is middleware in Express?
5. req.params vs req.query vs req.body?
6. How does JWT authentication work?
7. What are MongoDB indexes and why use them?
8. What are streams and Buffers?
9. How do you handle errors in an Express API?
10. How would you scale a Node.js application?

Final mental model

- Node.js = JavaScript runtime + event-driven I/O.
- Express = HTTP routing and middleware layer.
- Mongoose/MongoDB = persistence layer.
- Auth middleware = access-control boundary.
- Services = business logic.
- Streams, queues and workers = tools for large or expensive work.
- Production engineering = security + observability + reliability + scaling.